



通訊軟體ai機器人與asyncio學習筆記

科目：工程設計

學生：杜凱朗

授課老師：楊肅毅

我們這學期資訊課的期末作業是個人專題實作，我利用上課所學到的知識，完成了一個線上ai聊天機器人。

什麼是discord bot?

Discord 是目前最熱門且功能最全面的群組聊天軟體，但也因為功能全面，管理或是設定往往比較複雜。而Discord Bot 是一種自動化機器人，可在 Discord 伺服器上執行各種任務，如管理成員、播放音樂、提供遊戲資訊、執行自訂指令等。通常由 JavaScript、Python 或 Java 開發，並透過 Discord API 與伺服器互動。

常見功能

- 伺服器管理：自動分配身分組（分類每個成員權限的功能）、過濾訊息、踢出或禁言成員
- 音樂播放：從 YouTube、Spotify 播放音樂
- 娛樂與遊戲：迷因生成、小遊戲、抽獎活動
- 自動化與工具：提供天氣、新聞、翻譯、AI聊天機器人

其實就跟line機器人是一樣的道理，不過功能往往更強大，且上手開發更簡單。

我在上課中學到了什麼

一開始在進行期末專案的 Discord Bot 開發時，我遇到了一個嚴重的瓶頸：**當機器人呼叫 OpenAI API 生成回覆時，或是當我試圖同時啟動 Flask 網頁伺服器與 Discord Bot 時，整個程式就會被「卡住」（阻塞）**。在等待網路請求的這段期間，機器人完全無法接收或處理伺服器裡其他人的新訊息，導致使用體驗非常糟糕。

為了解決這個問題，我私下向精通 Python 的老師請教。老師向我介紹了 Python 的進階用法——**Asyncio**（非同步 I/O）**的概念與簡單應用。這是我第一次接觸到非同步程式設計，覺得非常新奇且剛好能解決我的痛點。於是，我隨後自己上網尋找資料深入研究（例如反覆觀看碼農高天的教學影片），並成功將 `asyncio` 完美應用到這次的期末專案作業中，徹底解決了程式阻塞的問題。

什麼是 asyncio ？

Asyncio 是 Python 標準庫內建的**非同步解決方案**，採用協程（coroutine）和事件迴圈（event loop）模型，能在不使用多執行緒或多進程的情況下實現非同步並發處理。然而，Asyncio 本質上仍是單執行緒，並不會真正提升運算速度，而是特別適用於需要等待的任務，典型的就網路通訊。

為什麼需要 asyncio ？

在傳統的同步程式中，如果程式遇到一個需要等候的動作（例如網路 I/O、檔案 I/O、或長時間計算），整個程式就會被這個動作「阻塞」，直到它完成為止。在這段阻塞期間，程式無法繼續處理其他事情。

asyncio主要應用在「大量 I/O 操作」或「需要同時處理很多事件」的場景下。它的優勢在於：

1. 不必透過多執行緒或多進程就能實現同時「處理多個工作」的效果（協同式多工）。
2. 提升 I/O 密集型工作的效率：當某個協程等待 I/O 時，事件迴圈可以轉而執行其他協程，不用浪費時間空等。

舉個簡單例子：

- 我需要同時和多個 API 通訊，每個 API 都要花 1 秒鐘回傳。如果是同步程式，一次只能呼叫一個 API，總共要花 5 秒（假設 5 個 API）。
- 若使用 **asyncio**，可以同時向 5 個 API 送出請求，只要 1 秒多的時間就能取得所有回應。

我的另一個專案：**ai文字遊戲**，就是一個例子，因為我有很多ai人物和ai環境判斷要執行，而我是使用openai的gpt4 api，假如我一個一個request，就會花數倍的時間，這時候使用 asyncio就能很好地解決問題。

核心概念

event loop

event loop 是Asyncio的核心，扮演著類似大腦的角色，面對眾多可執行的task時決定哪一個該先進行。由於 Python 同一時間只允許執行一個任務，因此每個任務必須自行通知 event loop 自己已經完成，實際上event loop不能主動停止哪個task，因此他有一個很大的好處是他沒有競爭冒險的問題，可以明確的知道每個任務的結束的先後順序。

協程(coroutine)與任務(task)

在 asyncio 中經常會提到 coroutine 與 task 這兩個概念。coroutine 包含了coroutine object和coroutine function，當呼叫一個 coroutine function 時，會返回一個coroutine object，但並不會立即執行其中程式碼，這有點像生成器，要讓這段程式碼運行，第一步是進入 async 模式，就是讓 event loop 控制程式的整體狀態，第二步是把coroutine object轉換成 task。一般來說，程式的入口函數是 asyncio.run，它的參數是一個 Coroutine object，這個函數會做兩件事，一是建立event loop，二是將參數coroutine object變成task，再把這個task當成event loop中的第一個task。另外定義coroutine function會用 `async def`。

await

Asyncio 最重要的功能就是處理多個task。而把coroutine object變成task有幾個方法，最常用的就是await，當await 一個coroutine function 時，function 會返回一個coroutine object，而這個object就會被變成task，並注冊進event loop。

Await後會有幾件事情發生

- 首先，他會把coroutine object變成task並告訴event loop這裡有一個新的task。
- 第二，告訴他現在這個正在執行的task要等到這個被await的完成後才能繼續，這樣就有了一個先後關係。
- 第三，他會把控制權還給event loop，因為他現在不能動了，要等別人，所以讓event loop選擇下一個要執行的task。
- 最後，當await等完，原本的函數繼續進行的時候，假如有，他會回傳被await的函數真正的返回值而不是coroutine object，然後保存起來。

其實就是字面上的意思，await就是等他後面的人，我一般寫沒那麼複雜的Asyncio都是這樣想的，event loop跟控制權的概念只有在自己寫函式的時候才需要特別注意。

協程函式範例

```
import asyncio

async def api_request(delay, say):
    await asyncio.sleep(delay)
    #這裡的asyncio.sleep也是coroutine, 會等待一秒, 不同的是他會釋出控制權
    print(say)

async def main():
    await api_request(1, "hello")
    await api_request(2, "world")

asyncio.run(main())
```

程式解釋

1. **Asyncio.run(main())**

- 呼叫main, 得到main的coroutine object
- asyncio.run建立event loop並把coroutine object變成event loop的第一個task

2. **main()**

- main() 執行到await api_request(1, "hello")
- api_request(1, "hello")變成task, 並宣告給event loop
- 釋出控制權給event loop, 最後開始等待
(這時event loop 中只有api_request(1, "hello")可以執行, 所以就執行它)

3. **api_request()**

- await asyncio.sleep(), api_request(1, "hello")釋出控制權, event loop執行asyncio.sleep(), 這時候event loop才進入真正的等待狀態 (因為沒有東西可以執行了)

4. **asyncio.sleep()**結束, event loop會發現api_request(1, "hello")可以繼續了, api_request結束後, event loop會發現main()可以繼續了, 有點像遞迴結束的撥洋葱, 一層一層撥開我的心。

5. 遇到api_request(2, "world")做跟前面一樣的事, 最後main()結束。

其實就是一連串的關係，誰等誰，等之前跟等完之後告訴event loop，然後event loop來決定執行誰。但實際運行下來後會發現程式花了3秒，這不是跟一般的sleep()一樣？

其實問題是出在第二個api_request()沒有先變成task註冊進event loop，程式中是等到第一個結束後才用await把他變成task並註冊的。想要發揮Asyncio的真正功能，我們需要提前註冊task，這時可以用asyncio的create_task函式來提前把coroutine object變成task並註冊進event loop。

程式範例

```
import asyncio

async def api_request(delay,say):
    await asyncio.sleep(delay)
    print(say)

async def main():
    task1 = asyncio.create_task(api_request(1,"hello"))
    task2 = asyncio.create_task(api_request(2,"world"))
    await task1
    await task2

asyncio.run(main())
```

程式解釋

這時候會發現程式只用了兩秒，且有先印出hello再印出world

其實流程就跟前面那個差不多，就是create_task提前把api_request(1,"hello")和api_request(2,"world")變成task並註冊進event loop

這裡有幾個地方我一開始沒有很懂，但理解後覺得滿酷的。

- 首先，為什麼create_task需要附值？
這是因為我們需要await task來知道每個task的先後順序和最後的回傳值。
- 再來，為什麼需要await兩次？ task不是都提前建立了？
這時候就要來再重新看一次流程：

1. main()
2. task1和2建立

3. `await task1`等待1秒，釋出控制權
4. `event loop`發現`task2`可以執行 執行`task2`等待兩秒
5. `task1`等待一秒結束，這時候`task2`也只等完一秒，控制權回到`main()`
6. 這時候運行到`await task2`，`task2`把剩下那一秒等完，程式才結束

但其實假如我只`await task2`也是一樣的效果，因為在等待`task2`的兩秒時`task1`的一秒可以等完，可是為了可讀性以及我們不一定能知道每個`task`的耗時，一般都會把`await`列出來或是使用 `Gather`

Gather

`Gather`是一個更進一步的工具，它返回一個叫`future`的東西。`Gather` 的參數可以是多個 `coroutine object`、`task`，甚至其他的`future`，我自己一般都是直接傳一個`tasks`的`task list`。若傳入的是 `coroutine object`，`Gather` 會先將它們轉換成 `task`並註冊到 `event loop` 中，然後返回一個 `future`。當我們 `await` 這個 `future` 時，就等於告訴 `event loop`必須等待其中所有的 `task` 完成後才繼續執行，最終返回的結果會以列表形式呈現。`gather`其實就等價於一個一個 `create_task`再一個個`await`。

asyncio與多執行緒/多進程的差別

- **多執行緒 (thread)**：適合 I/O 密集，但 Python 有 GIL 限制，CPU 密集的多執行緒不一定加速。且執行緒管理較複雜。
- **多進程 (process)**：每個進程都有獨立的 GIL，可以用於真正平行運算，但也需要較高的記憶體及資源成本，各進程之間的通訊也較複雜。
- **asyncio**：使用單執行緒 + `event loop`的模式，只要在 I/O 等待時就能切換到其他協程，是一種協同式多工。對 I/O 密集的應用非常有效，也比多執行緒或多進程更易管理與除錯。

小結

- 我個人覺得`asyncio`是保留傳統程式邏輯的一個很好的解決方案，我們不用管什麼`gil`或是系統層面的東西，一切都還在那個熟悉又簡單的`python`，雖然`asyncio`的概念乍看之下有點複雜，但只要把`event loop`跟控制權的概念放在腦海裡，在處理一些特定的問題時真的很方便。
- 其實大部分知識我都是在網路上自學而不是上課聽懂的，我參考了很多影片或是筆記，其中

最好的是碼農高天的影片，我敢說整個網路上就是他的教學最好理解且最詳細，我很多細節都是看完影片才懂的，核心概念的部分其實也可以當作我看完影片後的整理筆記，但包含了很多我自己學會後使用上的理解，以及我在其他地方找到的細節。

- 這個章節有點類似我的Asyncio學習筆記，我希望當我忘記概念的時候回來看可以看懂，我也會把他在hackmd(markdown筆記共享網站)公開，希望可以幫助到他人，所以這部分有點壟長還請見諒。
- 其實這個專題我使用的asyncio並不多，都是一些最基礎的await/async def而已，真正的玩轉asyncio是在前面提到的ai文字遊戲專案，在那我把asyncio玩出各種花樣，例如協調每個ai人物對環境的判斷的先後順序，環境事件的處理，以及ai人物記憶的處理，不過那個是千行程式以及數10個檔案的龐大專案，我的自主學習報告會專門介紹整個引擎的運作原理，而學習筆記我就放這邊了。

我使用到的技術

asyncio

- 主要幫助我處理非同步工作。在這個專案中，我同時需要啟動 Flask 服務以及 Discord Bot，而使用 `asyncio` 的 `run_in_executor` 就能把 Flask 用執行緒（非同步）方式在背景運行，並在同一個event loop中控制 Discord Bot 的執行。然後nextcord的核心就是用asyncio寫的，所以說我的函式想要能協程也要用asyncio

openai api (AzureOpenAI)

- 使用 AzureOpenAI SDK 或 OpenAI 的 API 與 AI 模型溝通，實現類似 ChatGPT 的對話功能。不使用官方api主要的原因是這個我有免費的.....

Flask

- 提供一個簡單的 HTTP 端點，讓這個專案可以在 PaaS（rander之類的雲端平台）上保持「應用程式存活」的狀態，也可以顯示簡單的狀態頁面，不會被當作閒置而被暫停。
- 其實這個是完全沒有必要的，我用他主要原因是rander只能免費託管網站，而假如我沒有開一個端口，就會被當作閒置而被暫停。另外當閒置太久被暫停時，我也可以連線這個ip的網頁來激活整個伺服器。

nextcord

- Discord Bot 的核心套件，用來監聽伺服器上的訊息、處理事件（例如 `on_ready`、`on_message`），並且註冊Slash Command等。其實原本的官方套件是 [discord.py](#)，但因為一些問題停止更新了，而nextcord算是discord.py的fork，繼承了大部分內容且有一些新東西比如slash command

dotenv

- 透過 `.env` 檔管理像是 `DISCORD_BOT_TOKEN`、`AZURE_OPENAI_API_KEY` 以及 `AZURE_OPENAI_ENDPOINT` 等資訊，減少資訊外洩的風險。
-

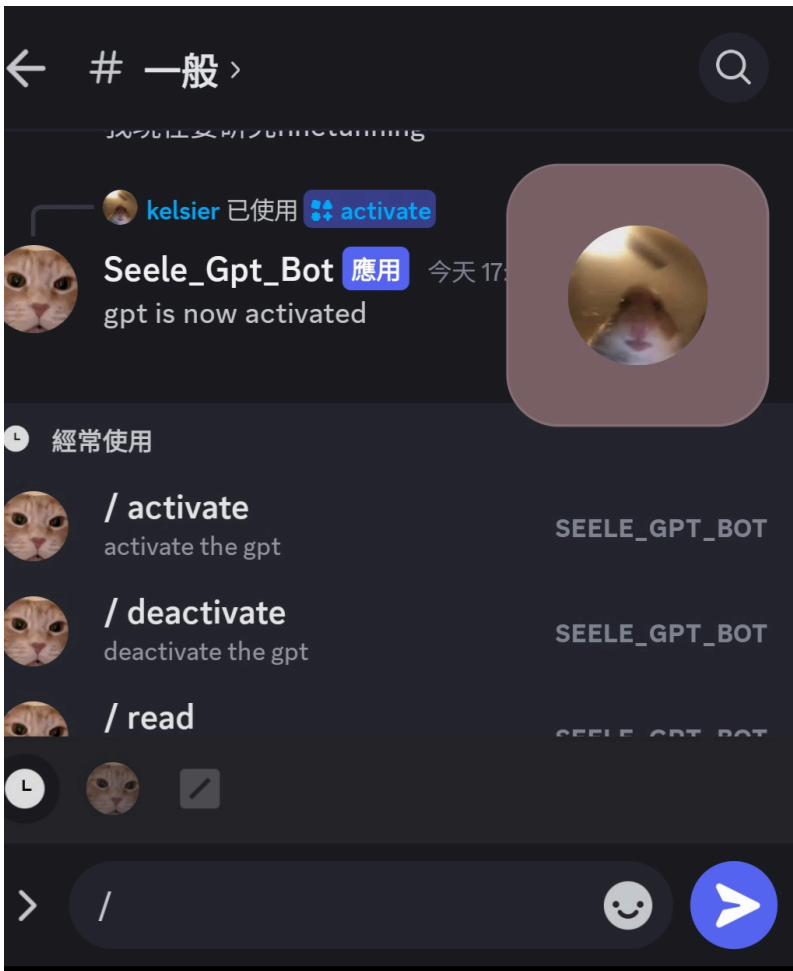
名詞解釋

- **cogs**

在 discord.py或是nextcord中，cogs 是一種模組化架構，讓我可以將機器人的命令和事件監聽器根據功能劃分到不同的類別，每個類別可以獨立管理、動態加載或卸載，這不僅讓代碼結構更加清晰、便於維護和擴展，也使得在開發和除錯過程中能夠快速定位和修改特定功能。

- **slash command**

slash command是discord 最新的api功能，可以把指令註冊進discord群組，一般來說我們使用dc bot都要像終端那樣打指令，而為了區分一般訊息跟指令，一般我們都會用前綴，而slash command就是使用 `/` 前綴，不同的是他會跳出快捷提示如下圖，而不用自己打完整個指令。



系統架構與技術流程

1. 環境變數載入

- 在程式啟動時，會使用 `dotenv` 載入 `.env` 檔案中的環境參數，包含 Discord Bot Token 與 OpenAI 的 API Key 等。

2. Flask 後台服務

- 執行 `run_flask_app()` 函式，在指定的 `port` 上啟動 Flask 應用程式。
- 路由 `/` 回傳簡單字串「Bot is running!」，做為存活檢測。

3. Discord Bot 啟動

- 透過 `nextcord` 套件建立 `Bot` 物件，並設定指令前綴、啟用 intents（如 `message_content` 等）。
- 事件 `on_ready` 觸發時，代表 Bot 已成功登入 Discord。
- 載入 `cogs`，將 GPT 相關邏輯模組化。

4. Cogs (GPT)

- 在 `gpt.py` 中，使用 `AzureOpenAI`（或 `openai` API）與 GPT 模型做互動。

- 每次有新的訊息到來時，會先檢查是否為自己 (Bot) 所發的訊息或目前功能是否啟用，再決定要不要回覆。
- 若偵測到圖片檔案，會透過 `requests` 下載圖片並轉成 base64 後，送到 GPT 以圖文混合的方式進行處理。
- 回覆完成後，再把 GPT 回覆也寫入訊息歷史 (`self.history`) 中。

5. 整合與非同步

- 使用 `asyncio.get_event_loop()` 建立事件迴圈，呼叫 `run_in_executor` 同時執行 Flask 服務，並在主執行緒使用 `run_until_complete` 啟動 Discord Bot。

實作細節與程式碼解析

主程式入口 (`main.py`)

```
cogs = ["gpt"]

async def run_discord_bot():
    for cog in cogs:
        bot.load_extension(f"cogs.{cog}")
    await bot.start(dc_token)

def run_flask_app():
    port = int(os.environ.get("PORT", 5000))
    app.run(host="0.0.0.0", port=port)

if __name__ == "__main__":
    loop = asyncio.get_event_loop()
    # 1. 在背景線程執行 Flask
    loop.run_in_executor(None, run_flask_app)
    # 2. 主線程啟動 Discord Bot
    loop.run_until_complete(run_discord_bot())
```

- `run_flask_app()` 會呼叫 `app.run()` 在背景啟動 Flask
- `run_discord_bot()` 內部則是載入 cogs 之後以 `bot.start(dc_token)` 登入 Discord

Discord Bot 基礎指令區

```
@bot.event
async def on_ready():
    print(f"目前登入身份 --> {bot.user}")

@bot.command()
async def load(ctx, extension):
    bot.load_extension(f"cogs.{extension}")
    await ctx.send(f"Loaded {extension} done.")

@bot.command()
async def unload(ctx, extension):
    bot.unload_extension(f"cogs.{extension}")
    await ctx.send(f"Unloaded {extension} done.")

@bot.command()
async def reload(ctx, extension):
    bot.reload_extension(f"cogs.{extension}")
    await ctx.send(f"Reloaded {extension} done.")
```

- 這些是寫在main.py的基礎指令，可以動態載入、卸載、重新載入指定的 cog，方便我在不重啟整個機器人的情況下進行更新以及測試。他的寫法跟使用方法也不同於slash command，寫法相對來說更簡單，使用上則是用\$作為前綴，也沒有自動補完跟提示，好處是方便且穩定。

GPT Cog ([gpt.py](#))

```
class Gpt(commands.Cog):
    guilds = [1270418744434*****, 11561169003219*****, 12924942919267*****]

    def __init__(self, bot):
        self.bot = bot
        self.gpt_state = True
        self.history = []

    @nextcord.slash_command(name="activate", description="activate the gpt", guild_ids=guilds)
    async def activate(self, interaction: Interaction):
        self.gpt_state = True
        await interaction.response.send_message("gpt is now activated")
```

- 以 slash command 來啟用或停用 GPT 功能，並且可以針對特定伺服器 (guild_ids) 使用。

```
@commands.Cog.listener()
async def on_message(self, message: nextcord.Message):
    if message.author == self.bot.user or not self.gpt_state:
        return
    # 偵測附件
    if message.attachments:
        # 處理圖片: 下載 -> base64 -> 傳給 GPT
        ...
    else:
        msgs = [
            {"role": "system", "content": f"以下是對話歷史記錄:{self.history}"},
            {"role": "user", "content": f"以下是最新訊息: time:{time} author:{message.author}"},
        ]
        # 呼叫 AzureOpenAI
        reply = str_request(msgs, 500)
        # 儲存回覆
        self.history.append(f"me: {reply}")
        # 實際發送訊息到頻道
        await message.channel.send(reply)
```

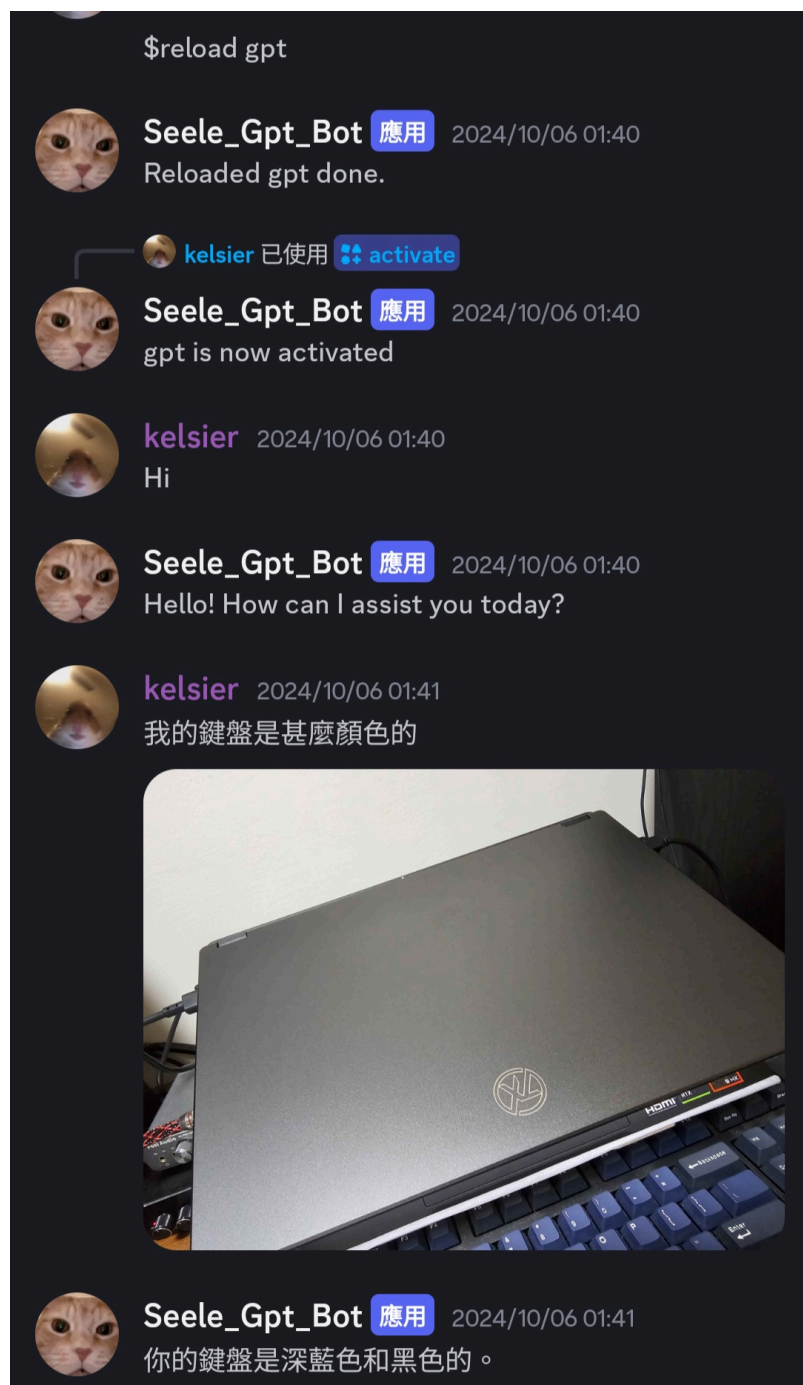
- 機器人會在 `on_message` 事件中對使用者訊息做出回應。
- 使用 `str_request` 與 OpenAI 互動，並將對話記錄（history）持續更新。

與 AzureOpenAI 互動

```
def str_request(messages, max_tokens):  
    try:  
        response = client.chat.completions.create(  
            model=deployment_name,  
            messages=messages,  
            max_tokens=max_tokens,  
            temperature=0.2  
        )  
        return response.choices[0].message.content  
    except:  
        return "error"
```

- 這裡把 GPT API 呼叫封裝到 `str_request` 函式中，保持主程式的簡潔。
-

成果展示



這裡首先我首先重新載入了gpt模組，bot也顯示重新載入成功，接下來我測試了基礎回覆功能，以及圖片功能



我遇到的困難

1. 非同步流程複雜度

一開始不熟悉如何同時在一個 Python 程式中同時啟動 Flask 伺服器與 Discord Bot。後來透過 `asyncio.run_in_executor` 在同一個事件迴圈中啟動兩個服務，才得以解決阻塞或程式先後執行的問題。

2. 訊息歷史的保留與管理

要讓 GPT 可以理解前後文，需要保存歷史訊息 (history)。一開始只是單純把聊天訊息都記下來，後來發現可能會累積過多且重複的訊息，於是增加了 `/read` 與 `/clear` 等指令手動控制讀取與清空。其實假如要正式公開應用我會需要使用資料庫，但後來就開始準備考試了，最近假如有時間的話我會試試。

3. azure openai

一開始我是用 request 的方式來跟 api 互動，但那程式真的又臭又長，後來我在網路上找到了 openai sdk 的使用方法，也搞定了可以上傳圖片的功能。

4. import discord

我最初是使用 discord.py 來寫這個專案的，但 discord.py 因為一些爭議而停止更新，我為了用上最新的功能轉向 nextcord

5. 圖片轉 base64

Discord 傳送的圖片必須要先下載、再轉成 base64，最後才給 GPT 處理。對於在 GPT Prompt 中嵌入圖片的情境需要額外嘗試。

6. 環境變數管理

為了在雲端部署時不外洩 Token，需要學習使用 `.env` 以及 `dotenv` 這些工具；另外我也在rander上設定環境變數卡關了一陣子。

心得與反思

- 我覺得是一個滿有趣的開發體驗，我也有確實學到了一些有用的東西，比如openai api的運用、nextcord、asyncio、rander雲端部署，也常常可以在我的其他專案看到他們的身影。還有這也是我第一次用markdown寫完整的學習筆記，我覺得沒有什麼花裡胡哨的東西，乾淨且易讀，效果非常好，我之後的學習歷程和課程學習成果應該都會用這個格式。
- 我在做這個專案時其實又犯了一個老毛病，我總是喜歡用最新的架構以及最新的技術，雖然能得到很酷的功能，但代價就是網路上的教學比較少，學習的時間成本比較高。網路上大部分的discord bot教學都是用一般前綴，因為slash command是很新的東西，且寫法跟架構都複雜很多，雖然最後我還是完成了這個專案，但除了我自己比較開心之外，大家只會看到一個快捷提示，不會知道這後面的程式多麼難搞。Cogs也是一樣的道理，他的架構跟最簡單的discord bot完全不一樣，一般連最大最熱門的bot都不一定採用這種架構，但我還是一股腦兒的更新了我原本的架構，除了更新時間，我也要額外花很多時間學習，最後換來的是我根本用不到的高維護性。為什麼說這是我的老毛病？其實在我的一些其他專案也可以看到我犯病，比如我的訂單管理網站我就從function base換成class base最後除了讓自己寫程式的效率變低，並沒有什麼好處。我每次使用套件時都會這樣，下次我會特別提醒自己，先看清自己的需求，再決定使用什麼架構。
- 最近在準備被審資料比較少寫程式了，不過假如有時間，我會繼續完善我的dc bot，以最終公開上架為目標，最好是可以讓我盈利賺點大學的零用錢，我腦袋裡也是有一些滿有趣的想法的，比如讓ai自動管理群組之類的，但老實說這也是一條很長的路，光是資料庫的部分就讓我頭大。
- 總的來說就是這樣，這是一個不錯的課程學習成果，有學到東西，也有做出來東西。

參考資料

碼農高天asyncio：<https://youtu.be/brYsDi-JajI?si=h6vpw5qQXDUHPdP6>

asyncio官方文檔：<https://docs.python.org/zh-tw/3.13/library/asyncio.html>

discord bot教學：<https://www.youtube.com/>

[watch?v=x7oBQNcNGeM&list=PLwqYQaS6jxfk_NCetUOyNRDGAf9_kU90n](https://www.youtube.com/watch?v=x7oBQNcNGeM&list=PLwqYQaS6jxfk_NCetUOyNRDGAf9_kU90n)

專案github repo：https://github.com/Kelsier64/gpt_discord_bot